

Problem Set — Week 2

Intractability and Heuristics · VCE Algorithmics (HESS) · Unit 4 AOS2

Sections A–E correspond to Topics 7–11. Attempt each after its lesson.

Name: _____

Due: Tuesday 28 July

Vocabulary warning: in this unit **SAT** is used for two unrelated things. In Section B it means *Boolean satisfiability*, a problem in logic. Everywhere else in the course it means your *School-Assessed Task*. This paper writes “satisfiability” for the problem and spells out School-Assessed Task in full.

Section A — Tractability, P and NP

(Topic 7)

A1. Define *tractable* and *intractable*, and explain why the dividing line is drawn at polynomial time rather than at some particular speed.

A2. Write the **decision version** of each optimisation problem.

- a.) Find the shortest tour visiting every city.
- b.) Find the largest set of pairwise non-adjacent nodes in a graph.
- c.) Find the timetable with the fewest clashes.

A3. Define the classes **P** and **NP**, state the relationship between them, and explain why that relationship holds.

A4. A student writes: “Intractable problems are ones where there is no way of checking a solution quickly, and so it is impossible to know whether they are decidable.” Identify **two** distinct errors and give an improved definition.

A5. You buy a machine 1000 times faster than your current one. Explain what this does for a problem solvable in $O(n^3)$ time, and for one requiring $O(2^n)$ time. Use your answer to justify why exponential growth is treated as a barrier rather than an inconvenience.

Section B — NP-completeness, reductions, satisfiability

(Topic 8)

B1. *Boolean satisfiability.* For each formula, state whether it is satisfiable. If it is, give a satisfying assignment; if not, explain briefly how you know.

a.) $(p \vee \neg q) \wedge (q \vee r) \wedge (\neg p \vee \neg r)$

b.) $(p) \wedge (\neg p \vee q) \wedge (\neg q)$

B2. Explain why Boolean satisfiability is in NP. Your answer must describe the polynomial-time check explicitly.

B3. State the **two** conditions a problem must meet to be NP-complete, and say which one alone gives NP-hardness.

B4. You want to prove a new problem X is NP-hard. You have a known NP-complete problem K . Which reduction do you construct — $K \leq_p X$ or $X \leq_p K$? Justify your choice, and state what the wrong direction would actually establish.

B5. The Halting Problem is NP-hard but *not* NP-complete. Explain why.

B6. The 0/1 knapsack problem has a dynamic programming solution running in $O(nW)$ time, where W is the capacity. Explain why this does **not** make knapsack tractable. *Hint: how many bits are needed to write down the number W ?*

Section C — Dynamic programming and backtracking

(Topic 9)

C1. State the two properties a problem must have for dynamic programming to be worthwhile, and describe what goes wrong if each is absent.

C2. A coin system has denominations $\{1, 5, 8\}$. You need to make 15.

- Write the recurrence $C(v)$ for the fewest coins making value v , including its base case.
- Fill the table below.
- State the optimal number of coins and which coins they are.
- How many coins does a greedy (largest-first) approach use? Explain why greedy fails here.

v		0	1	2	3	4	5	6	7	8	9	10	11	12	13	14	15
$C(v)$		0															

C3. Exam-style. A backtracking algorithm abandons a branch as soon as a partial candidate violates a constraint. Many complete candidates are therefore never generated. Explain why the algorithm nevertheless still finds the optimal solution.

C4. Classify each pruning rule as **sound** (the optimum cannot be lost) or **unsound**, with a one-line reason.

- a.) "This partial tour is already longer than the best complete tour found so far." (All edge weights positive.)
- b.) "This partial tour is longer than average, so abandon it."
- c.) "This partial assignment already places two queens on the same diagonal."

C5. Does backtracking improve the *worst-case* complexity of an exponential problem? Justify.

Section D — Greedy algorithms and hill climbing

(Topic 10)

D1. Define what a *heuristic* is, being precise about what is given up and what is gained.

D2. Describe a greedy approach to the travelling salesman problem. Your answer must identify what makes the approach greedy, not merely describe the steps.

D3. Before running hill climbing you must specify three things. Name them, and give a concrete choice for each in the context of TSP.

D4. Is Dijkstra's algorithm a heuristic? Justify your answer by reference to the definition in D1.

D5. Explain why hill climbing cannot detect that it has stopped at a local rather than a global optimum. Name two techniques that reduce the risk, and state why neither is a guarantee.

Section E — Simulated annealing

(Topic 11)

E1. Explain how a simulated annealing algorithm decides whether to accept a new candidate solution. Cover both the improving and the worsening case, and state what the acceptance probability depends on.

E2. State **one valid** terminating condition and **one invalid** one, explaining why the second cannot be used.

E3. A student has implemented simulated annealing correctly, but finds it only ever accepts candidates that improve on the current solution. Describe how the **parameters** should be changed, and explain the symptom.

E4. Describe one advantage and one limitation of using simulated annealing for TSP.

E5. Explain why a correct implementation returns the *best solution seen* rather than the solution held when the loop terminates.

Section F — Connecting to your School-Assessed Task

F1. Identify the bottleneck module of your Unit 3 solution — the one whose complexity dominates. Then argue **either**:

- a.) that a heuristic should replace it — naming the representation, the neighbourhood and the objective function you would use, and stating what guarantee you give up; **or**
- b.) that an exact technique (dynamic programming, backtracking, or a better algorithm choice) fits better — naming it and justifying why the exact route remains feasible.
